18、esp8089 wifi驱动

全志V3S嵌入式驱动开发（esp8089 wifi驱动）

原创

嵌入式-老费Open: 7f185b501ef38a059b10d1b79d7cfdb9_MD5.png

已于 2023-08-29 15:02:50 修改

Open: 8a923f94c34ac151897c6be747452d7c_MD5.png

阅读量3k Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏 7

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数 1

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

摘要 本文详细介绍了在全志V3S平台上进行ESP8089 WiFi模块的驱动开发过程，包括原理图分析、设备树修改、驱动文件准备、wpa_supplicant编译以及测试步骤。通过该过程，读者可以了解到Linux环境下WiFi驱动的加载和测试方法。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

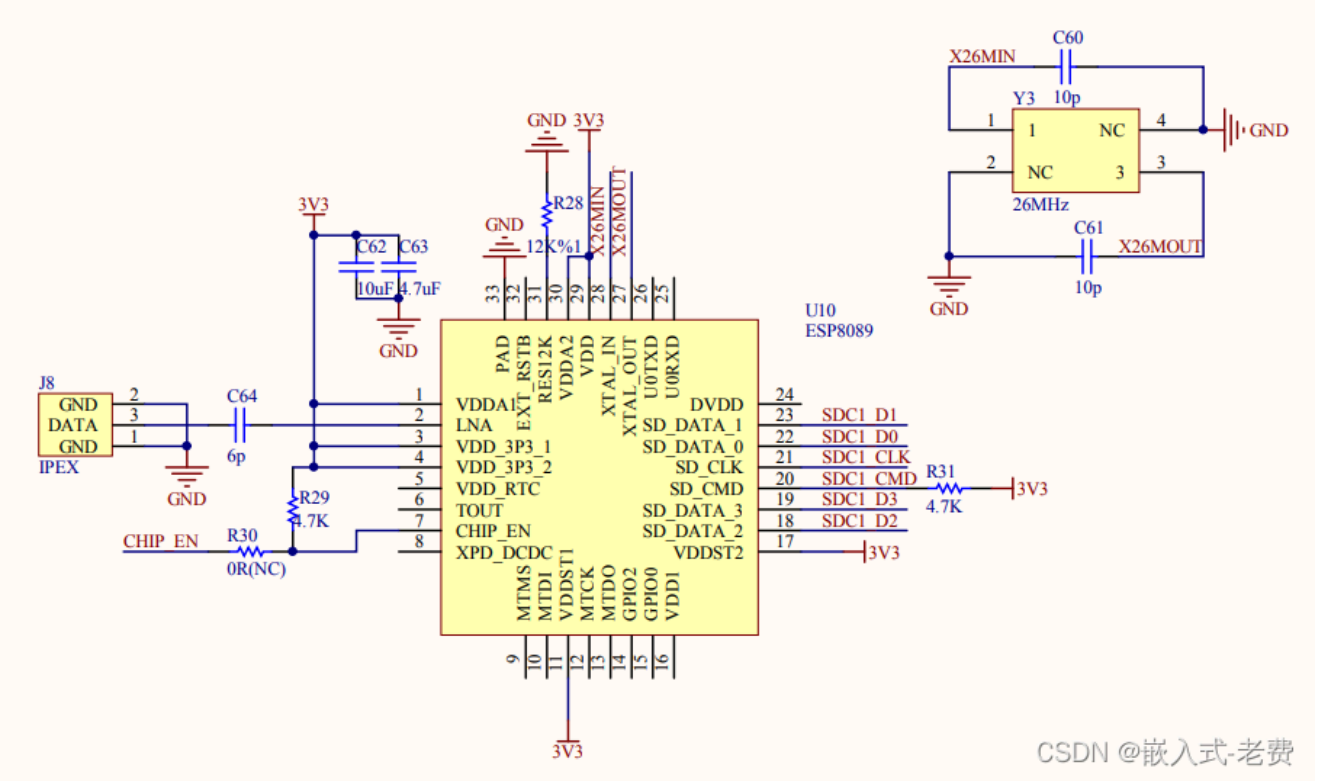
【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing @163.com】

荔枝派上面除了支持v3s自带的有线网口，还带有一个支持sd协议的esp 8089 wifi模块。有了这个模块，v3s没有网线，也可以用它和外界联系了。但是，我们实际测试下来，效果并不算很理想。这其中最主要的原因，就是网络延迟有点高，哪怕路由器就在测试桌旁，改善也不是很大。当然，我们也看到有人使用了wifi模块RTL8723BS，不知道效果如何。有使用经验的朋友不妨留个言，相互学习下。

当然，不管esp8089自身质量如何，都不妨碍我们了解一下整个开发过程，比如在Linux下面，一个WiFi设备驱动应该如何加载，测试应该如何进行。

1、原理图

Open: 5c9675fc0f60af9f5d30cfbdf71f5662_MD5.png



开始今天的讨论之前，我们不妨重新看一下这个wifi电路图。首先最上面是两个晶振的引脚，右上方是26M晶振的输入输出图。右侧是SDC信号，除了clk和cmd之外，就是d0-d3。下方除了一个3v3，其他都是悬空。左侧除了一个ipex天线，剩下来就是一个CHIP_EN，通看全图的话，发现它和PWM0复用同一个信号。

Open: 5ba54f8dbd9a3b335620cc28a80aee28_MD5.png

2、修改dtsi文件

看完了原理图，下面第一个要准备的就是修改设备树文件。修改的地方都位于同一个文件，即sun8i-v3s-licheepi-zero-with-lcd.dtsi。首先，关闭led，因为和wifi模块有冲突，

```
    leds {
        /* The LEDs use PG0~2 pins, which conflict with MMC1 */
```

```
        status = "disbaled";
    };
```

其次，使能mmc1，也就是这个wifi接口，

```
&mmc1 {
    broken-cd;
    bus-width = <4>;
    vmmc-supply = <&reg_vcc3v3>;
    status = "okay";
};
```

有同学如果想要对pin引脚和寄存器、中断了解更多，可以参考文件sun8i-v3s-licheepi-zero.dts中的内容。 **编译好之后，记得把新的dtb文件拷贝到sd卡上面。**

3、准备驱动文件

目前在linux 4.14.y上面没有看到esp 8089的驱动，所以这部分需要自己去其他地方下载。比如，有一个下载的地址位于github这个地方，即

```
https://github.com/al177/esp8089
```

下载好之后，可以拷贝到ubuntu虚拟机，准备开始编译。编译方法很多，可以选用这种，直接赋值KBUILD和SRC_DIR，

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
KBUILD=~/Desktop/linux-zero-4.14.y SRC_DIR=~/Desktop/esp8089-master
```

或者含蓄一点，使用-C和M来代替，也是可以的

```
ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- make -C ~/Desktop/linux-
zero-4.14.y M=${PWD}  modules
```

不管哪一种方法，其实都是可以生成esp8089.ko文件的，

```
feixiaoxing@feixiaoxing-VirtualBox:~/Desktop/esp8089-master$ make
make -C /usr/src/linux-headers-5.15.0-73-generic
M=/home/feixiaoxing/Desktop/esp8089-master
make[1]: Entering directory '/usr/src/linux-headers-5.15.0-73-generic'
./scripts/pahole-flags.sh: line 7: return: can only `return' from a
```

```
function or sourced script
./scripts/pahole-flags.sh: line 7: return: can only `return' from a
function or sourced script
./scripts/pahole-flags.sh: line 7: return: can only `return' from a
function or sourced script
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/esp_debug.o
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/sdio_sif_esp.o
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/spi_sif_esp.o
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/esp_io.o
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/esp_file.o
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/esp_main.o
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/esp_sip.o
/home/feixiaoxing/Desktop/esp8089-master/esp_sip.c: In function
‘sip_recalc_credit_timeout’:
/home/feixiaoxing/Desktop/esp8089-master/esp_sip.c:220:24: warning:
cast to pointer from integer of different size [-Wint-to-pointer-cast]
  220 |  struct esp_sip *sip = (struct esp_sip *)list->flags;
      |                        ^
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/esp_ext.o
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/esp_ctrl.o
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/esp_mac80211.o
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/esp_utils.o
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/esp_pm.o
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/testmode.o
  LD [M]  /home/feixiaoxing/Desktop/esp8089-master/esp8089.o
./scripts/pahole-flags.sh: line 7: return: can only `return' from a
function or sourced script
./scripts/pahole-flags.sh: line 7: return: can only `return' from a
function or sourced script
  MODPOST /home/feixiaoxing/Desktop/esp8089-master/Module.symvers
  CC [M]  /home/feixiaoxing/Desktop/esp8089-master/esp8089.mod.o
  LD [M]  /home/feixiaoxing/Desktop/esp8089-master/esp8089.ko
  BTF [M] /home/feixiaoxing/Desktop/esp8089-master/esp8089.ko
Skipping BTF generation for /home/feixiaoxing/Desktop/esp8089-
master/esp8089.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-5.15.0-73-generic'
```

编译好之后，就可以把es8089.ko拷贝到sd卡的root目录下，等待准备驱动安装了。

4、编译wpa_supplicant

wpa_supplicant是wifi配置一个非常常见的工具，某种意义上来说，近乎于标配。它对于wifi模块的意义，就和tslib对于触摸屏的意义是一样的。此外，wpa_supplicant又依赖于libnl和openssl这两个库，所以我们需要把这三个文件一起下载下。

有同学也许会说，为什么不用buildroot去尝试编译wpa_supplicant，省的自己麻烦。主要是我们确实去尝试了，无奈编译出错，所以只好又搭建交叉编译环境来生成wps_supplicant文件了。

```
https://ftp.openssl.org/source/old/1.0.2/openssl-1.0.2.tar.gz
https://www.infradead.org/~tgr/libnl/files/libnl-3.2.23.tar.gz
https://w1.fi/releases/wpa_supplicant-2.9.tar.gz
```

三个文件的链接地址如上所述。首先编译libnl，

```
tar xzvf libnl-3.2.23.tar.gz
cd libnl-3.2.23/
mkdir __install
./configure --host=arm-linux-gnueabihf --
prefix=/home/feixiaoxing/Desktop/wpa/libnl-3.2.23/__install/
make
make install
```

其次，编译openssl，它的configure有一点特殊，主要是os/compiler这个标签，大家注意下，

```
tar xzvf openssl-1.0.2.tar.gz
cd openssl-1.0.2/
mkdir __install
./Configure os/compiler:arm-linux-gnueabihf-gcc --
prefix=/home/feixiaoxing/Desktop/wpa/openssl-1.0.2/__install/
make
make install
```

准备好了libnl和openssl之后，就可以开始编译wpa_supplicant了。当然，因为它还依赖于libnl和openssl，所以必要的解压和环境准备是少不了的，

```
tar xzvf wpa_supplicant-2.9.tar.gz
cd wpa_supplicant-2.9/wpa_supplicant
cp defconfig .config
```

在.config种需要修改这几个变量

```
CC=arm-linux-gnueabihf-gcc
CFLAGS += -I/home/feixiaoxing/Desktop/wpa/libnl-
3.2.23/__install/include
CFLAGS += -I/home/feixiaoxing/Desktop/wpa/openssl-
1.0.2/__install/include
LIBS += -L/home/feixiaoxing/Desktop/wpa/libnl-3.2.23/__install/lib
LIBS += -L/home/feixiaoxing/Desktop/wpa/openssl-1.0.2/__install/lib
```

另外，.config中还需要注销DBUS的内容，不然编译有错误，

```
# Add support for new DBus control interface
# (fi.w1.hostap.wpa_supplicant1)
#CONFIG_CTRL_IFACE_DBUS_NEW=y

# Add introspection support for new DBus control interface
#CONFIG_CTRL_IFACE_DBUS_INTRO=y
```

最后一步，就是重新设置PKG_CONFIG_PATH，开始编译，

```
export PKG_CONFIG_PATH=/home/feixiaoxing/Desktop/wpa/libnl-
3.2.23/__install/lib/pkgconfig:$PKG_CONFIG_PATH
make
```

如果一切顺利，编译结束后，我们就可以看到wpa_supplicant和wpa_cli这两个我们需要的可执行文件了。 **前面我们把dtb文件拷贝好了，也把驱动esp8089.ko拷贝到root目录下，这里同样也需要把wpa_supplicant和wpa_cli拷贝到root目录下。另外，因为wpa_supplicant依赖libnl的动态库文件，建议把libnl-3.2.23/__install/lib/目录下面的所有文件拷贝到root目录下，这样就省得麻烦了。**

5、准备测试

前面我们已经把esp8089.ko和两个wpa软件、动态库拷贝进来。这个时候还要继续设置一下环境变量，这样wpa_supplicant才使用地起来。

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/root
```

首先安装esp8089.ko驱动， **这个43就是CHIP_EN引脚，**

```
# insmod esp8089.ko esp_reset_gpio=43
[   37.298395] esp8089: loading out-of-tree module taints kernel.
[   37.307097]
[   37.307097] ***** EAGLE DRIVER VER:bdf5087c3deb*****
[   37.307097]
[   37.315229] ESP8089 reset via GPIO 43
[   37.534706] esp_sdio_dummy_probe enter
[   37.754493] esp_sdio_init power up OK
[   38.209435] esp_host:bdf5087c3deb
[   38.209435] esp_target: e826c2b3c9fd 57 18202
[   38.209435]
[   38.218699] esp_readwrite_file: file
/system/lib/modules/test_results filp_open error
[   38.227448] first normal exit
[   38.230608] esp_sdio_remove enter
[   38.234122] sif_disable_irq release irq failed
[   38.354675] eagle_sdio: probe of mmc1:0001:1 failed with error -110
# [   38.724858] mmc1: card 0001 removed
[   38.819146] mmc1: queuing unknown CIS tuple 0x01 (3 bytes)
[   38.831765] mmc1: queuing unknown CIS tuple 0x1a (5 bytes)
[   38.840496] mmc1: queuing unknown CIS tuple 0x1b (8 bytes)
[   38.847837] mmc1: queuing unknown CIS tuple 0x80 (1 bytes)
[   38.853361] mmc1: queuing unknown CIS tuple 0x81 (1 bytes)
[   38.859313] mmc1: queuing unknown CIS tuple 0x82 (1 bytes)
[   38.864893] mmc1: new high speed SDIO card at address 0001
[   39.228942] esp_host:bdf5087c3deb
[   39.228942] esp_target: e826c2b3c9fd 57 18202
[   39.228942]
```

之后，就可以用ifconfig让wlan0上线了，

```
ifconfig wlan0 up
```

wlan0虽然准备好了，但是还没有ssid和密码，这部分需要配置下，即准备/etc/wpa_supplicant.conf这个文件，

```
ctrl_interface=/var/run/wpa_supplicant
network={
    ssid="xiaoyan_home"
```

```
    psk="88888888"
}
```

这里psk用的是明文，大家可以根据自己的情况做一个修改。如脚本所示，当然还需要创建一个wpa_supplcant的目录，即，

```
mkdir -p /var/run/wpa_supplicant
```

此外，因为我们测试的时候，有线网口和无线网口来自同一个路由器，为了减少不必要的麻烦，我们先关闭eth0，为测试做准备，

```
ifconfig eth0 down
```

所有这些都做好之后，就可以开始测试wifi了。主要的工作流程也是两步，第一步就是连上wifi，这个工作由wpa_supplicant完成，ssid和密码位于/etc/supplicant.conf文件，

```
./wpa_supplicant -D wext -c /etc/wpa_supplicant.conf -i wlan0 &
```

第二步就是用ifconfig对wlan0进行ip设置，注意这里就没有使用udhcpc命令了，大家自己测试的时候也可以换成udhcpc，一般都是busybox自带的，

```
ifconfig wlan0 192.168.0.231 netmask 255.255.255.0 broadcast
192.168.0.1
```

使用udhcpc就是这样的，

```
# udhcpc -i wlan0
udhcpc: started, v1.27.1
udhcpc: sending discover
udhcpc: sending select for 192.168.0.104
udhcpc: lease of 192.168.0.104 obtained, lease time 7200
deleting routers
adding dns 192.168.1.1
adding dns 192.168.0.1
```

如上面脚本所示，ip被设置为了231，这个时候pc地址是100，而且当前开发板和外界唯一的通信就是wifi模块，所以可以看下ping的质量如何，

Open: d40b61b92bfad3fa0ef2631e48fe955e_MD5.png

```
# ifconfig wlan0 192.168.0.231 netmask 255.255.255.0 broadcast 192.168.0.1
# ping 192.168.0.100
PING 192.168.0.100 (192.168.0.100): 56 data bytes
64 bytes from 192.168.0.100: seq=0 ttl=128 time=88.905 ms
64 bytes from 192.168.0.100: seq=1 ttl=128 time=9.848 ms
64 bytes from 192.168.0.100: seq=2 ttl=128 time=10.587 ms
64 bytes from 192.168.0.100: seq=3 ttl=128 time=48.441 ms
64 bytes from 192.168.0.100: seq=4 ttl=128 time=71.594 ms
64 bytes from 192.168.0.100: seq=5 ttl=128 time=181.556 ms
64 bytes from 192.168.0.100: seq=6 ttl=128 time=16.777 ms
64 bytes from 192.168.0.100: seq=7 ttl=128 time=81.755 ms
64 bytes from 192.168.0.100: seq=8 ttl=128 time=64.000 ms
64 bytes from 192.168.0.100: seq=9 ttl=128 time=87.934 ms
64 bytes from 192.168.0.100: seq=10 ttl=128 time=50.487 ms
64 bytes from 192.168.0.100: seq=12 ttl=128 time=1080.515 ms
64 bytes from 192.168.0.100: seq=13 ttl=128 time=80.864 ms
64 bytes from 192.168.0.100: seq=15 ttl=128 time=2073.412 ms
64 bytes from 192.168.0.100: seq=17 ttl=128 time=1096.686 ms
64 bytes from 192.168.0.100: seq=18 ttl=128 time=97.252 ms
64 bytes from 192.168.0.100: seq=19 ttl=128 time=205.452 ms
64 bytes from 192.168.0.100: seq=20 ttl=128 time=42.004 ms
64 bytes from 192.168.0.100: seq=21 ttl=128 time=65.570 ms
64 bytes from 192.168.0.100: seq=22 ttl=128 time=180.058 ms
64 bytes from 192.168.0.100: seq=23 ttl=128 time=197.856 ms
64 bytes from 192.168.0.100: seq=24 ttl=128 time=34.439 ms
64 bytes from 192.168.0.100: seq=25 ttl=128 time=58.147 ms
64 bytes from 192.168.0.100: seq=26 ttl=128 time=81.789 ms
^C
--- 192.168.0.100 ping statistics ---
27 packets transmitted, 24 packets received, 11% packet loss
round-trip min/avg/max = 9.848/250.247/2073.412 ms
```

CSDN @嵌入式-老费

测试结果看，延时的起伏还是比较大的。这说明，本身模块的功能，只是解决有无的问题，可以在小范围使用，如果是大规模出货的话，还是要斟酌一下的。

此外程序运行一段时间，哪怕是ping的时候，也可能崩溃的，这个就有点不爽了。

```
[   42.711009] Unable to handle kernel paging request at virtual
address 7365636d
[   42.718293] pgd = c0004000
[   42.720997] [7365636d] *pgd=00000000
[   42.724600] Internal error: Oops: 5 [#1] SMP ARM
[   42.729211] Modules linked in: esp8089(O)
[   42.733226] CPU: 0 PID: 55 Comm: kworker/0:2 Tainted: G           O
4.14.14-licheepi-zero #10
[   42.741996] Hardware name: Allwinner sun8i Family
[   42.746710] Workqueue: events_freezable mmc_rescan
[   42.751495] task: c39fc000 task.stack: c3b8a000
[   42.756058] PC is at esp_sdio_probe+0x4c/0x3f4 [esp8089]
[   42.761368] LR is at sdio_bus_probe+0x108/0x11c
[   42.765890] pc : [<bf00117c>]    lr : [<c04e9fe4>]    psr: 200b0013
[   42.772147] sp : c3b8bdf0  ip : 00000000  fp : 0000034e
[   42.777363] r10: 0000000c  r9 : bf00a188  r8 : 73656369
[   42.782581] r7 : bf00d078  r6 : c3191000  r5 : bf039744  r4 :
c3199e80
[   42.789097] r3 : c3191008  r2 : 00000002  r1 : 00000000  r0 :
```

```
c3191000
[   42.795616] Flags: nzCv  IRQs on  FIQs on  Mode SVC_32  ISA ARM
Segment none
[   42.802740] Control: 10c5387d  Table: 431a006a  DAC: 00000051
[   42.808476] Process kworker/0:2 (pid: 55, stack limit = 0xc3b8a210)
[   42.814734] Stack: (0xc3b8bdf0 to 0xc3b8c000)
[   42.819088] bde0:                                     bf00a188
00000000 c3191008 00000000
[   42.827256] be00: c3191000 bf00d0dc bf00a188 00000000 0000000c
c04e9fe4 c3191008 c0a87104
[   42.835423] be20: c0a87108 bf00d0dc 00000000 c0415fc0 00000000
c3b8be68 c0416130 00000001
[   42.843591] be40: c0a870e0 00000000 00000000 c04145d0 c391cb6c
c3199ab8 c3191008 c319103c
[   42.851758] be60: c0a3d3a4 c0415cac c3191008 00000001 c3191008
c3191010 c3191008 c0a3d3a4
[   42.859926] be80: 00000000 c0415280 c3191010 c3301808 c3191008
c0413748 00000001 c04e0820
[   42.868093] bea0: c3191000 c3191008 c3191000 c3191008 c3b72800
c3301800 c3301b5c c3301800
[   42.876261] bec0: 00000001 c04ea27c 00000001 00000001 c3b72800
c04e971c 00000000 c3b8beeb
[   42.884429] bee0: 00000000 00000000 00000000 10ffff00 00000000
c3b72a54 00061a80 c3b72800
[   42.892596] bf00: c0759358 c0759364 00000000 c3b8a000 c3f66640
c04e13b0 c3b72a54 c3b7f880
[   42.900764] bf20: c3f66640 00000000 c3f69800 c0131cd0 00000008
c0a02d00 c3b7f880 c3f66640
[   42.908931] bf40: c3b7f898 00000008 c0a02d00 c3f66658 c3b8a000
c0132984 00000000 c3b77dc0
[   42.917099] bf60: ffffe000 c3b8c080 00000000 c3b77dc0 c3b8a000
c3b7f880 c013294c c383bee0
[   42.925266] bf80: c3b8c09c c01377b4 c3b8a000 c3b77dc0 c0137690
00000000 00000000 00000000
[   42.933433] bfa0: 00000000 00000000 00000000 c0107868 00000000
00000000 00000000 00000000
[   42.941600] bfc0: 00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000
[   42.949767] bfe0: 00000000 00000000 00000000 00000000 00000013
00000000 00000000 00000000
[   42.957962] [<bf00117c>] (esp_sdio_probe [esp8089]) from
```

```
[<c04e9fe4>] (sdio_bus_probe+0x108/0x11c)
[   42.966923] [<c04e9fe4>] (sdio_bus_probe) from [<c0415fc0>]
(driver_probe_device+0x234/0x2e8)
[   42.975444] [<c0415fc0>] (driver_probe_device) from [<c04145d0>]
(bus_for_each_drv+0x60/0x94)
[   42.983963] [<c04145d0>] (bus_for_each_drv) from [<c0415cac>]
(__device_attach+0xb0/0x114)
[   42.992221] [<c0415cac>] (__device_attach) from [<c0415280>]
(bus_probe_device+0x84/0x8c)
[   43.000390] [<c0415280>] (bus_probe_device) from [<c0413748>]
(device_add+0x3f4/0x580)
[   43.008300] [<c0413748>] (device_add) from [<c04ea27c>]
(sdio_add_func+0x5c/0x70)
[   43.015776] [<c04ea27c>] (sdio_add_func) from [<c04e971c>]
(mmc_attach_sdio+0x26c/0x33c)
[   43.023858] [<c04e971c>] (mmc_attach_sdio) from [<c04e13b0>]
(mmc_rescan+0x360/0x3ec)
[   43.031686] [<c04e13b0>] (mmc_rescan) from [<c0131cd0>]
(process_one_work+0x1ec/0x414)
[   43.039598] [<c0131cd0>] (process_one_work) from [<c0132984>]
(worker_thread+0x38/0x5b4)
[   43.047682] [<c0132984>] (worker_thread) from [<c01377b4>]
(kthread+0x124/0x154)
[   43.055075] [<c01377b4>] (kthread) from [<c0107868>]
(ret_from_fork+0x14/0x2c)
[   43.062291] Code: e3a01000 e5851018 e2863008 e5948004 (e5981004)
[   43.068455] ---[ end trace 2d229c26e46b073e ]---
```

16、总结

esp8089这个驱动，可能是目前位置我们遇到的、最复杂的驱动。当然，这也没关系。学会了它，要是以后再遇到类似的驱动，相信也不会比它更复杂了，我们也心里更有底了。有兴趣的同学可以好好看下这个过程，争取活学活用，为进一步的开发工作打下基础。